Solve Using MapReduce: LinkedIn Profile Views

We have data:

ID,From-Member,To-Member

1,Manasa,Sumit

2,Deepa,Sumit

3,Sumit,Manasa

4,Manasa,Deepa

5,Deepa,Manasa

6,Shilpy,Manasa

We need:

- How many people viewed **Sumit's** profile
- How many viewed **Manasa's** profile
- How many viewed **Deepa's** profile

Understanding Data

Each row means:

From-Member viewed To-Member profile

Example:

Manasa,Sumit

means:

Manasa viewed Sumit's profile

So we count using **To-Member**.

MapReduce Logic

Mapper Phase

Read each row and emit:

(To-Member,1)

Mapper Output Row by Row

Row 1

1,Manasa,Sumit

Output:

(Sumit,1)

Row 2

2,Deepa,Sumit

Output:

(Sumit,1)

Row 3

3,Sumit,Manasa

Output:

(Manasa,1)

Row 4

4,Manasa,Deepa

Output:

(Deepa,1)

Row 5

5,Deepa,Manasa

Output:

(Manasa,1)

Row 6

6,Shilpy,Manasa

Output:

(Manasa,1)

Full Mapper Output

(Sumit,1)

(Sumit,1)

(Manasa,1)

(Deepa,1)

(Manasa,1)

(Manasa,1)

Shuffle and Sort Phase

Hadoop groups same keys:

Sumit -> [1,1]

Manasa -> [1,1,1]

Deepa -> [1]

Reducer Phase

Add values.

Sumit

1 + 1 = 2

Manasa

1 + 1 + 1 = 3

Deepa

1 = 1

Final Output

Sumit 2

Manasa 3

Deepa 1

Answer

Profile	Views
Sumit	2
Manasa	3

Profile	Views
Deepa	1

How MapReduce Solves It

Map Step

Convert each record into:

(profile_viewed,1)

Reduce Step

Add all counts for each profile.

One-Line Logic

Count how many times each person appears in **To-Member** column.

Real Hadoop Thinking

If this file had 10 crore rows:

- Many mappers read chunks in parallel
- Reducers sum counts by member name

That's why MapReduce is powerful.

Pseudocode

Mapper

emit(ToMember,1)

Reducer

sum(values)

emit(member,total_views)

Final One-Line Answer

MapReduce uses the **To-Member** as key, emits (name,1), then reducer sums values to get total profile views.

MapReduce Internal Process (LinkedIn Profile Views)

We'll use your data:

ID,From-Member,To-Member

1,Manasa,Sumit

2,Deepa,Sumit

3,Sumit,Manasa

4,Manasa,Deepa

5,Deepa,Manasa

6,Shilpy,Manasa

Goal:

Count profile views for:

- Sumit
- Manasa
- Deepa

First Understand What Each Row Means

From-Member viewed To-Member profile

Example:

Manasa viewed Sumit

So we count **To-Member** values.

Full Internal Hadoop Flow

Input File in HDFS

↓

Input Split

↓

RecordReader

↓

Mapper

↓

Intermediate Key-Value

↓

Shuffle & Sort

↓

Reducer

↓

Final Output in HDFS

Step 1: File Stored in HDFS

Suppose this CSV file is stored in Hadoop HDFS.

Large files are split into blocks.

Since your file is tiny, one block is enough.

So one mapper can process it.

Step 2: InputSplit Created

Hadoop creates logical split for mapper.

Split 1 = whole file

If huge file:

many splits → many mappers.

Step 3: RecordReader Reads File

RecordReader reads one line at a time and gives:

(key, value)

Where:

- key = line byte offset
- value = complete line text

Example:

(0,"1,Manasa,Sumit")

(15,"2,Deepa,Sumit")

...

Step 4: Mapper Starts Processing

Mapper ignores ID and From-Member.

It uses:

To-Member

and emits:

(To-Member,1)

Mapper Row-by-Row Internally

Input Row 1

1,Manasa,Sumit

Mapper Output:

(Sumit,1)

Input Row 2

2,Deepa,Sumit

Output:

(Sumit,1)

Input Row 3

3,Sumit,Manasa

Output:

(Manasa,1)

Input Row 4

4,Manasa,Deepa

Output:

(Deepa,1)

Input Row 5

5,Deepa,Manasa

Output:

(Manasa,1)

Input Row 6

6,Shilpy,Manasa

Output:

(Manasa,1)

Full Mapper Output

(Sumit,1)

(Sumit,1)

(Manasa,1)

(Deepa,1)

(Manasa,1)

(Manasa,1)

Step 5: Partitioning

If multiple reducers exist:

Hadoop decides which reducer gets which key.

Example:

Reducer 1 -> Sumit

Reducer 2 -> Manasa

Reducer 3 -> Deepa

Uses hash of key internally.

Step 6: Shuffle Phase

Now Hadoop moves same keys together across network.

This is called:

Shuffle

It collects all same names.

Step 7: Sort + Group

Hadoop sorts keys and groups values:

Deepa -> [1]

Manasa -> [1,1,1]

Sumit -> [1,1]

Reducer receives grouped data like this.

Step 8: Reducer Runs

Reducer logic:

sum all values

For Deepa

Deepa -> [1]

Total = 1

For Manasa

Manasa -> [1,1,1]

Total = 3

For Sumit

Sumit -> [1,1]

Total = 2

Step 9: Final Output Written to HDFS

Deepa 1

Manasa 3

Sumit 2

Stored in:

part-00000

Real Internal Java-Style Logic

Mapper

Read line

Split by comma

Take 3rd column

Emit(name,1)

Reducer

For each name:

  sum counts

Emit(name,total)

Why This Is Powerful for Huge Data

If file had **100 crore profile views**:

- many blocks
- many mappers process in parallel
- reducers combine totals

Fast and scalable.

Key Internal Components

Component	Work
HDFS	Stores file
RecordReader	Reads lines
Mapper	Emits (ToMember,1)
Shuffle	Groups same names
Reducer	Adds counts
OutputFormat	Saves result

Memory Trick

To-Member = key

1 = count

Reducer = sum

Final Answer

MapReduce internally reads each record, extracts viewed profile name (To-Member), emits (name,1), groups same names during shuffle, reducer sums counts, and writes final profile views.

Final Output

Profile	Views
Sumit	2
Manasa	3
Deepa	1

Multiple Mappers + 2 Reducers Simulation

LinkedIn Profile Views — Exactly How Hadoop Distributes Work

We'll assume **much more data** exists.

Input Data

ID,From,To

1,Manasa,Sumit

2,Deepa,Sumit

3,Sumit,Manasa

4,Manasa,Deepa

5,Deepa,Manasa

6,Shilpy,Manasa

7,Ravi,Sumit

8,Kiran,Deepa

9,Ajay,Manasa

10,Nita,Sumit

11,Priya,Deepa

12,John,Manasa

Goal:

Count profile views.

Step 1: File Stored in HDFS

Suppose Hadoop splits file into **3 blocks**.

Block1 -> rows 1 to 4

Block2 -> rows 5 to 8

Block3 -> rows 9 to 12

So Hadoop creates:

- Mapper1 → Block1
- Mapper2 → Block2
- Mapper3 → Block3

Cluster Example

3 machines:

NodeA stores Block1

NodeB stores Block2

NodeC stores Block3

Hadoop runs mapper near data (**data locality**).

Step 2: Mapper Execution

Mapper1 (Block1)

Rows:

1 Manasa -> Sumit

2 Deepa -> Sumit

3 Sumit -> Manasa

4 Manasa -> Deepa

Output:

(Sumit,1)

(Sumit,1)

(Manasa,1)

(Deepa,1)

Mapper2 (Block2)

Rows:

5 Deepa -> Manasa

6 Shilpy -> Manasa

7 Ravi -> Sumit

8 Kiran -> Deepa

Output:

(Manasa,1)

(Manasa,1)

(Sumit,1)

(Deepa,1)

Mapper3 (Block3)

Rows:

9 Ajay -> Manasa

10 Nita -> Sumit

11 Priya -> Deepa

12 John -> Manasa

Output:

(Manasa,1)

(Sumit,1)

(Deepa,1)

(Manasa,1)

Combined Mapper Output

Sumit 1,1,1,1

Manasa 1,1,1,1,1

Deepa 1,1,1

But not yet grouped physically.

Step 3: Partitioning to 2 Reducers

Now Hadoop decides:

- Reducer1 gets some keys
- Reducer2 gets remaining keys

Uses:

hash(key) % number_of_reducers

Assume:

Reducer1 -> Sumit, Deepa

Reducer2 -> Manasa

Exact Distribution from Each Mapper

Mapper1 Sends

To Reducer1:

(Sumit,1)

(Sumit,1)

(Deepa,1)

To Reducer2:

(Manasa,1)

Mapper2 Sends

To Reducer1:

(Sumit,1)

(Deepa,1)

To Reducer2:

(Manasa,1)

(Manasa,1)

Mapper3 Sends

To Reducer1:

(Sumit,1)

(Deepa,1)

To Reducer2:

(Manasa,1)

(Manasa,1)

This Network Movement = Shuffle

All mapper outputs travel to correct reducer.

This is costly and important stage.

Step 4: Sorting & Grouping at Reducers

Reducer1 Receives

(Sumit,1)

(Sumit,1)

(Sumit,1)

(Sumit,1)

(Deepa,1)

(Deepa,1)

(Deepa,1)

Grouped:

Sumit -> [1,1,1,1]

Deepa -> [1,1,1]

Reducer2 Receives

(Manasa,1)

(Manasa,1)

(Manasa,1)

(Manasa,1)

(Manasa,1)

Grouped:

Manasa -> [1,1,1,1,1]

Step 5: Reducers Calculate Final Count

Reducer1

Sumit = 4

Deepa = 3

Reducer2

Manasa = 5

Final Output Files

Reducer1 writes:

part-00000

Deepa 3

Sumit 4

Reducer2 writes:

part-00001

Manasa 5

Final Combined Result

Profile	Views
Sumit	4
Manasa	5
Deepa	3

Full Hadoop Flow

Large File

↓

3 Blocks

↓

3 Mappers (parallel)

↓

(name,1)

↓

Partition by hash

↓

Shuffle over network

↓

2 Reducers

↓

Output files

Why Multiple Reducers?

Benefits:

- faster final aggregation
- workload split
- scalable for huge keys

Why Same Key Must Go to Same Reducer?

Because all counts for:

Sumit

must meet in one place to total correctly.

Interview-Level Insight

Mappers scale by input size

Reducers scale by number of unique keys / aggregation load

Honest Real-World Truth

In billion-row data:

- hundreds of mappers
- dozens of reducers

Same exact logic.

Final One-Line Answer

Hadoop splits large data into blocks, runs multiple mappers in parallel, hashes each key to one of 2 reducers, shuffles matching keys together, reducers sum counts, and write separate output files.

example if file size is 1gb and we have 3 cluster node then how it will work

Example: 1 GB File + 3 Cluster Nodes — How Hadoop MapReduce Works

Let's assume:

- File size = **1 GB**
- Cluster nodes = **3 machines**
- Hadoop block size = **128 MB**

We'll see exactly how it works.

Step 1: File Stored in HDFS

1 GB = 1024 MB

Number of blocks:

$$\frac{1024}{128} = 8$$

So Hadoop splits file into **8 blocks**.

B1 B2 B3 B4 B5 B6 B7 B8

Step 2: Blocks Distributed Across 3 Nodes

Suppose:

Node1 -> B1 B4 B7
Node2 -> B2 B5 B8
Node3 -> B3 B6

(Actual placement may vary)

Usually blocks also have replicas.

Step 3: MapReduce Job Submitted

You run:

hadoop jar job.jar

Hadoop checks:

- where blocks are stored
- available CPU/memory
- free task slots

Step 4: One Mapper per Block

Usually:

1 block = 1 mapper

So:

8 blocks = 8 mappers

Mapper list:

- M1 → B1
- M2 → B2
- M3 → B3
- M4 → B4
- M5 → B5
- M6 → B6
- M7 → B7
- M8 → B8

Step 5: Data Locality (Very Important)

Hadoop tries to run mapper on same node where block exists.

Example:

Node1 runs M1 for B1
Node2 runs M2 for B2
Node3 runs M3 for B3

Why?

Because moving code is easier than moving data.

Step 6: Parallel Execution

Suppose each node can run **2 mappers at a time**

Then:

Round 1

Node1 -> M1(B1), M4(B4)
Node2 -> M2(B2), M5(B5)
Node3 -> M3(B3), M6(B6)

6 mappers run together.

Round 2 (after some finish)

Node1 -> M7(B7)
Node2 -> M8(B8)

Now remaining 2 mappers run.

Step 7: Mapper Output

Suppose word count job.

Mapper reads records and emits:

(word,1)

Each mapper creates intermediate data from its block.

Step 8: Shuffle Phase

All mapper outputs are sent across nodes to reducers.

Same keys go together.

Example:

apple -> all counts together
data -> all counts together
hadoop -> all counts together

Step 9: Reducers Run

Suppose 2 reducers.

Reducer1 -> A-M words
Reducer2 -> N-Z words

They sum values.

Example:

apple = 5000
data = 9000
hadoop = 1200

Step 10: Final Output

Reducers write result back to HDFS:

part-00000
part-00001

Full Visual Flow

1 GB File
↓
8 Blocks
↓
Stored on 3 Nodes
↓
8 Mappers
↓
Parallel processing
↓
Shuffle
↓
Reducers
↓
Output

What If One Node Fails?

Suppose Node2 crashes.

Hadoop uses replicated blocks from other nodes and reruns failed tasks.

That is fault tolerance.

Why This Is Faster Than One Machine

Instead of one system reading 1 GB alone:

- 3 machines process different parts together
- work happens in parallel

Real Example Timing Idea

Method	Time
One machine	Slow
3 nodes parallel	Much faster

(Depends on hardware/job type)

Key Concepts from This Example

Concept	Meaning
HDFS Blocks	File split into chunks
Mapper	Processes one block
Data Locality	Run code where data exists
Shuffle	Move grouped keys
Reducer	Final combine

Final One-Line Answer

A 1 GB file becomes 8 blocks, blocks are spread across 3 nodes, Hadoop runs 8 mappers near the data in parallel, then reducers combine results and store final output.

Honest Tip

For interviews, remember:

 # Mappers ≈ number of input splits
not number of nodes.

---

step-6 explain more neatly

Step 6: Parallel Execution — Explained Neatly

We had:

- **1 GB file**
- **8 blocks**
- **8 mappers**
- **3 cluster nodes**

Now Step 6 means:

How Hadoop runs those 8 mapper tasks across 3 machines at the same time.

First Understand This

A node (machine) can run **limited tasks at one time** depending on:

- CPU cores
- RAM
- YARN resources
- Configuration

Suppose each node can run:

2 mapper tasks simultaneously

We have 3 nodes.

So total parallel mappers at one time:

$$3 \text{ nodes} \times 2 = 6 \text{ mappers at once}$$

So Hadoop can run **6 mappers together**.

We Need to Run Total 8 Mappers

Because:

B1 B2 B3 B4 B5 B6 B7 B8

Each block = one mapper.

So:

M1 M2 M3 M4 M5 M6 M7 M8

Round 1 Execution

Hadoop starts first 6 mappers immediately.

Node	Running Tasks
Node1	M1(B1), M4(B4)
Node2	M2(B2), M5(B5)
Node3	M3(B3), M6(B6)

So:

- 6 blocks are being processed now
- 2 mappers still waiting

Waiting:

M7(B7), M8(B8)

What Are These Mappers Doing?

Each mapper:

- reads its block
- processes records
- emits intermediate output

Example for word count:

(data,1)
(hadoop,1)
(big,1)

After Some Time...

Suppose:

- M1 finished
- M3 finished

Now two slots become free.

So Hadoop starts waiting tasks.

Round 2 Execution

Node	Running Tasks
Node1	M7(B7)
Node3	M8(B8)

Now all 8 mappers have run.

Why Not Run All 8 At Once?

Because cluster has resource limit.

Only 6 mapper slots available at that moment.

So Hadoop runs tasks in **waves / rounds**.

Very Simple Visual

Round 1:
M1 M2 M3 M4 M5 M6

Round 2:
M7 M8

Real Hadoop Logic

Scheduler checks:

- free memory
- free CPU
- data locality
- queue priority

Then assigns next mapper.

Why This Is Powerful

Instead of processing 8 blocks one after another:

B1 → B2 → B3 → ... → B8

Hadoop processes many together:

B1 B2 B3 B4 B5 B6 simultaneously

Much faster.

If Each Mapper Takes 10 Minutes

Sequential (1 machine)

$$8 \times 10 = 80 \text{ minutes}$$

Parallel (6 first, then 2)

Round 1 = 10 min
Round 2 = 10 min

Total ≈ **20 minutes** (simplified)

Huge improvement.

Important Interview Point

Number of mappers is based on input splits, but number running simultaneously depends on cluster resources.

Final One-Line Answer

Step 6 means Hadoop runs as many mapper tasks in parallel as cluster resources allow, and remaining tasks wait until slots become free.

Honest Advice

This scheduling concept is core distributed systems thinking. Understand it deeply—not just “8 blocks = 8 mappers.” That’s beginner level; concurrency limits are the real understanding.